Automated Intelligent Trash Collector Bin

MCTR 1010 Image Processing For Mechatronics

Hadil Amin, Mostafa Maged, Youssef Mohamed, Ahmed Ashraf

March 17, 2026

Abstract

This document presents the literature review and initial design for an automated intelligent trash collector bin. The system uses a Raspberry Pi 4B, Kinect camera, Mecanum wheels and an ESP32 microcontroller to catch thrown objects and return to a docking station when full. Key image processing techniques include depth sensing, object detection, gesture recognition, trajectory prediction using a Kalman filter and AprilTag-based navigation. Preliminary work on geometric transformations, intensity adjustments, Gaussian smoothing, head detection and red color detection is also described.

1 Introduction

This project is about building a smart motorized trash bin that can catch objects thrown at it and then drive itself to a docking station when it gets full. The system is built around a Raspberry Pi 4B as the main processor, a Kinect camera for depth sensing and vision, a Mecanum wheel chassis driven by four geared DC motors and two H-bridges, and an ESP32 microcontroller that acts as a bridge between the Raspberry Pi and all the physical components. When the bin is full, a limit switch on the inside of the lid triggers the system to lock the lid using a servo motor and begin the return-to-base routine. The user can also give voice commands using a built-in voice recognition module, or use hand gestures detected by the camera, to trigger the catching mode.

This literature review covers six research papers, that are directly relevant to the key parts of our image processing pipeline: using the Kinect camera for depth and object detection, recognizing gestures with MediaPipe, predicting the flight path of the thrown object using a Kalman filter, navigating back to the docking station using AprilTag markers, and moving the bin accurately in any direction using Mecanum wheels.

2 Preliminary Image Processing Techniques

Before implementing the main object tracking and gesture recognition pipeline, we conducted several foundational image processing experiments to build familiarity with core computer vision concepts and establish reliable preprocessing routines for the Raspberry Pi environment.

2.1 Geometric Transformations: Rotation and Scaling

In any real-world deployment, the orientation and size of objects in the camera frame can vary significantly depending on where the user stands and how they throw the object. To ensure the system remains robust to these variations, we implemented geometric transformation functions that can rotate and scale regions of interest within the camera image. Rotation corrects for angled trajectories if the object enters the frame at an unusual orientation due to the thrower's position, the system can mathematically rotate the bounding box to align it with the expected orientation for detection. Scaling ensures that objects at different distances appear at consistent sizes in the detection pipeline: an object close to the camera will appear larger and must be scaled down, while an object farther away will appear smaller and can be scaled up to match the expected input size of our detector. This preprocessing step ensures that the subsequent detection algorithms receive visually consistent input regardless of the object's position or orientation in the 3D space.

2.2 Intensity Level Transformation: Brightening and Contrast Adjustment

Lighting conditions in indoor environments are rarely perfect. Shadows from overhead lights, glare from windows, or simply the time of day can all affect how clearly the camera sees the thrown object. To address this, we implemented intensity level transformations that adjust the brightness and contrast of incoming frames before they enter the detection pipeline. Brightening is particularly useful in low-light situations where the object might otherwise blend into the background. By applying a simple intensity scaling factor to each pixel, we can lift dark regions of the image while preserving the relative differences between objects. More sophisticated contrast adjustments spread out the most frequent intensity values across the full available range, making textures and edges more visible. In our tests, these intensity transformations significantly improved detection rates under non-ideal lighting, especially during evening hours when indoor lighting becomes more variable.

2.3 Smoothing Algorithm: Gaussian Filter

Raw camera images always contain some degree of noise. This noise can confuse object detection algorithms, causing them to see false edges or miss real ones. To clean up the image before further processing, we apply a Gaussian filter, which smooths the image by averaging each pixel with its neighbors, weighted by a Gaussian distribution. Pixels closer to the center contribute more to the average than pixels farther away, preserving the overall structure of objects while blurring out high-frequency noise. The Gaussian filter runs as the very first step in our image processing pipeline on every frame, ensuring that all subsequent operations receive the cleanest possible input.

2.4 Object-Specific Detection: Head Detection and Red Color Detection

As part of our exploration into lightweight detection methods suitable for the Raspberry Pi, we developed two specialized detectors.

Head Detection: The head detector serves two purposes: first, it helps the system ignore the user's movements when determining whether an object has been thrown (avoiding false triggers when the user simply waves or shifts position) and second, it provides a reference point for where the throw originated, which can be used as an initial guess in the Kalman filter prediction.

Red Color Detection: As a simple but effective fallback detection method, we implemented a color-based detector that identifies regions of the image with high red content.

3 The Kinect Camera for Depth Sensing and Object Detection

Instead of using two separate USB webcams to estimate depth (as in a traditional stereo setup), our project uses an Xbox Kinect camera. The Kinect is an RGB-D sensor, meaning it captures both a regular color image and a depth image at the same time. The depth image tells the system how far away every pixel in the frame is, without needing to do any stereo matching calculations. This makes it much simpler and faster to get 3D position data compared to a dual-camera approach.

A 2022 study by Annese et al. evaluated the depth accuracy of the Kinect v2 and the newer Azure Kinect across different distances and lighting conditions [1]. The study found that the Kinect produces reliable depth measurements at ranges of 0.5 to 4.5 meters, with depth error staying well under a few centimeters at short to medium distances. This

range is more than enough for our bin, since the user will typically be throwing objects from 1 to 3 meters away. The study also confirmed that the Kinect's depth data works well with OpenCV, which means calibration and integration are straightforward on the Raspberry Pi 4B.

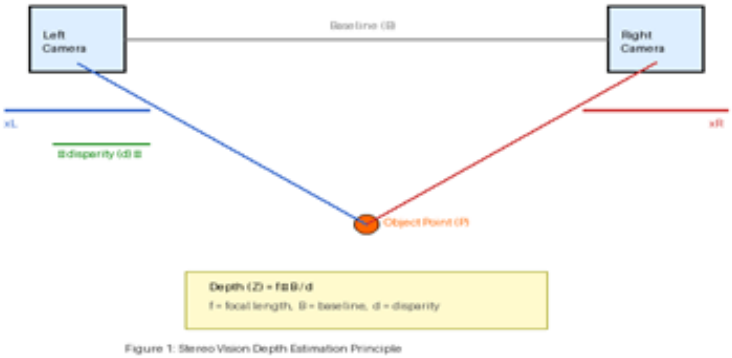

Figure 1: How the Kinect generates depth data. Unlike stereo cameras that compare two images, the Kinect uses an infrared sensor to directly measure the distance to each point in the scene, outputting a depth map alongside the RGB image.

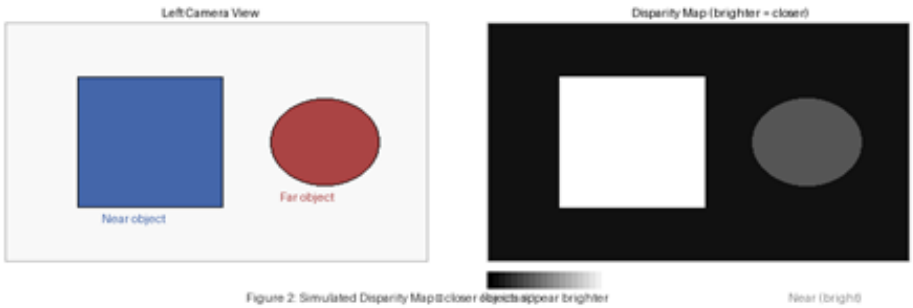

Figure 2: An example depth map. Brighter pixels are closer objects; darker pixels are farther away. The Kinect provides this map directly at up to 30 frames per second, giving the system real-time 3D position data without any extra processing.

4 Object Detection and Gesture Recognition

The system needs to do two different kinds of visual recognition: detecting the gesture that tells the bin to get ready, and then detecting and tracking the thrown object once it is in the air. A 2024 study by Baduge et al. tested several deep learning object detection models on edge devices including the Raspberry Pi 4 [2]. They found that YOLOv8 nano gives a good balance between speed and accuracy on low-power hardware without needing a GPU. Based on this, we plan to use YOLOv8 nano for the initial object detection step when the bin is in waiting mode.

For tracking the object once it is thrown, we switch away from any neural network and instead use a classical method called background subtraction. This works by comparing each new camera frame to a stored image of the empty background, anything that has moved shows up highlighted. This method runs in just a couple of milliseconds per frame, which is fast enough to track the object throughout its short flight without overloading the Raspberry Pi. The Kinect's depth channel is then used to extract the 3D position of that region at each frame, giving us a full 3D tracking point at every step.

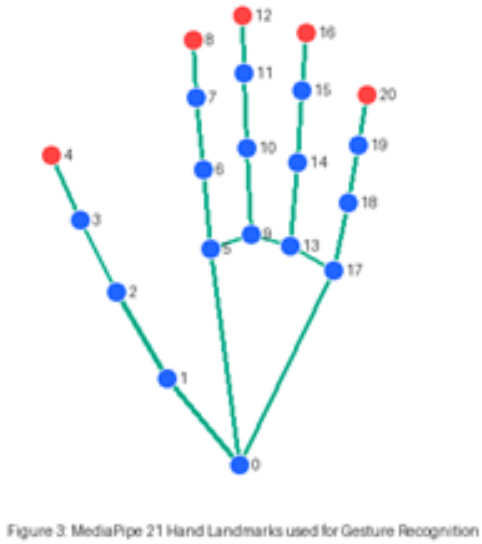

Figure 3: The 21 hand landmarks detected by MediaPipe Hands [3]. Each number is a specific joint or fingertip. By checking which fingers are extended or folded, the system can classify gestures like an open hand (ready to catch) or a closed fist (cancel).

For gesture recognition, we use MediaPipe Hands, a tool developed by Google (Zhang et al., 2020) that detects 21 key points on the hand in real time directly from the camera image [3]. It was built to run on mobile phones and small computers without needing a GPU, so it works well on the Raspberry Pi 4B. In our project, gesture recognition only runs when the bin is idle. The moment the system detects that an object has been thrown, it switches to the tracking pipeline and stops running gesture detection, so the two tasks never compete for processing power at the same time.

5 Predicting Where the Object Will Land

Once the object is thrown, the system needs to predict where it will land before it gets there, so the bin has time to drive into position. A thrown object follows a curved path

through the air governed by basic physics — gravity pulls it down at a constant rate while it moves forward. If we measure the object's 3D position in the first few frames after it is thrown, we can plug those values into simple kinematic equations to calculate the landing spot. The Kinect makes this easier than a stereo camera setup because we get accurate depth right away without any calibration or matching step.

The raw position readings from the Kinect will still have some small noise and jitter. A Kalman filter fixes this by combining what the sensor is telling us (the noisy measurements) with what physics tells us (how an object under gravity should be moving) to produce a smooth, accurate estimate of the object's position and speed at every frame. A 2022 paper by Jacewicz et al. specifically tested Kalman filter variants for estimating projectile trajectories and showed that even with significant measurement noise, the filter could reliably predict the impact point from a short sequence of observations [4]. This directly supports our plan to use a Kalman filter on the Kinect depth measurements to predict the landing coordinates in real time.

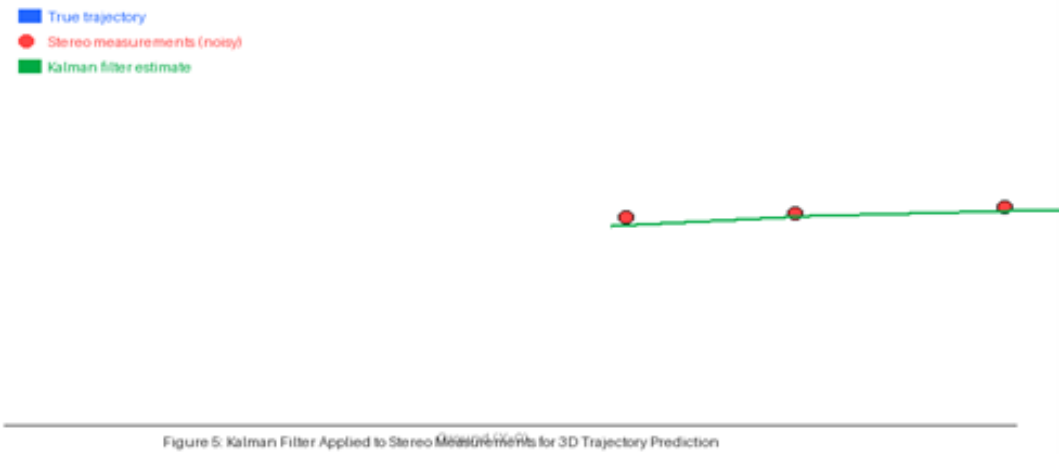

Figure 4: How the Kalman filter works in our system. The red dots are the noisy 3D position readings from the Kinect. The green line is the smooth filtered estimate of the flight path. The orange zone marks the predicted landing spot that the bin will move to.

6 Finding and Docking at the Base Station

When the bin is full, the limit switch inside the lid triggers the system to stop catching and return to the base station. To dock accurately, we attach a printed AprilTag marker to the base station wall. AprilTags are black-and-white square patterns that a camera can detect instantly. When the Kinect spots one, it can calculate how far away the tag is, what direction it is in, and what angle it is being viewed from, all from a single frame. The tag itself is just printed on paper, so it costs almost nothing to set up.

A 2025 paper by Zeng et al. built a full autonomous docking system for mobile robots that used YOLOv8 to detect AprilTags and then guided the robot into a precise

docking position [5]. Their system showed that AprilTag detection stays reliable even when lighting changes or the tag is partially covered. For our bin, the approach will be split into two steps: first the bin estimates its position using wheel odometry to drive roughly toward the station, then it switches to AprilTag-based guidance for the final precise alignment.

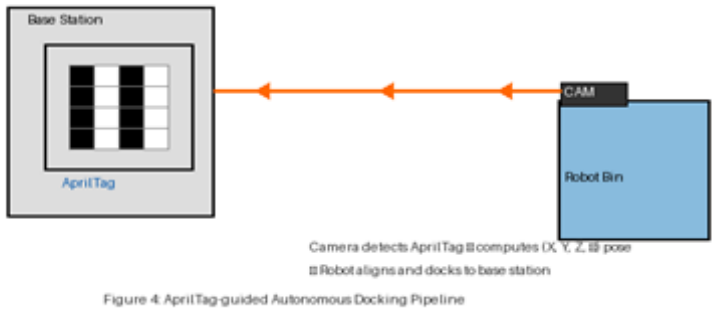

Figure 5: How the docking process works. The Kinect camera spots the AprilTag on the base station, calculates the distance and angle, and the bin uses that information to steer itself into the correct docking position.

7 Moving in Any Direction with Mecanum Wheels

One of the key advantages of our design is the use of Mecanum wheels. A standard robot can only move forward, backward, or spin in place. Mecanum wheels have small rollers attached at a 45-degree angle around the rim, so by controlling the speed and direction of each of the four wheels independently, the robot can move in any direction without rotating at all. This is important for our bin because the thrown object could land anywhere around it, not just directly in front.

A 2024 study by Shafiq et al. designed and tested a Mecanum-wheeled mobile robot for navigation in dynamic indoor environments [6]. Their paper worked out the kinematic equations that describe how four wheel speeds translate into robot motion in any direction, and validated these on a real robot. They confirmed that with four DC motors each controlled by an H-bridge, a Mecanum robot can achieve smooth omnidirectional movement with simple speed commands. This matches our setup exactly, four geared DC motors, two H-bridges (each controlling two motors), and a Mecanum wheel on each corner, with the ESP32 converting velocity commands from the Raspberry Pi into PWM signals for the H-bridges.

8 How All the Parts Work Together

The Raspberry Pi 4B runs the main image processing pipeline. When idle, it processes the Kinect's RGB stream through MediaPipe to look for a gesture, or listens for a voice command from the voice recognition module. Once triggered, it switches to tracking mode, using the Kinect's depth stream to get 3D positions and feeding them into the Kalman filter to predict the landing zone. The predicted coordinates are sent to the ESP32 over serial, which converts them into motor commands for the H-bridges driving the Mecanum wheels. When the limit switch detects the bin is full, the ESP32 triggers the servo to lock the lid and signals the Pi to start the docking routine, which uses AprilTag detection from the Kinect to guide the bin home.

The reason for splitting control between the Pi and the ESP32 is that the Pi is good at running image processing and AI tasks but is not ideal for real-time motor control with precise timing. The ESP32 handles the low-level tasks like PWM generation and reading sensors, while the Pi focuses purely on vision. This split keeps both processors working within their strengths.

9 Initial System Design

Based on the component choices described above and the findings from the six papers reviewed, the team developed two initial design documents: a wiring diagram showing how all electronic components are connected, and a SolidWorks 3D model showing the physical layout of the bin. These are presented below as the starting point for the hardware implementation.

9.1 Wiring Diagram

The wiring diagram below shows the connections between all major electronic components. The Raspberry Pi 4B is connected to the ESP32 over serial (UART). The ESP32 is then connected to two L298N H-bridge motor driver modules — one controlling the two left Mecanum wheels and one controlling the two right Mecanum wheels. A push button (SW1) is also connected to the ESP32 for manual override. All signal routing goes through the ESP32, keeping the Raspberry Pi free to focus entirely on image processing tasks.

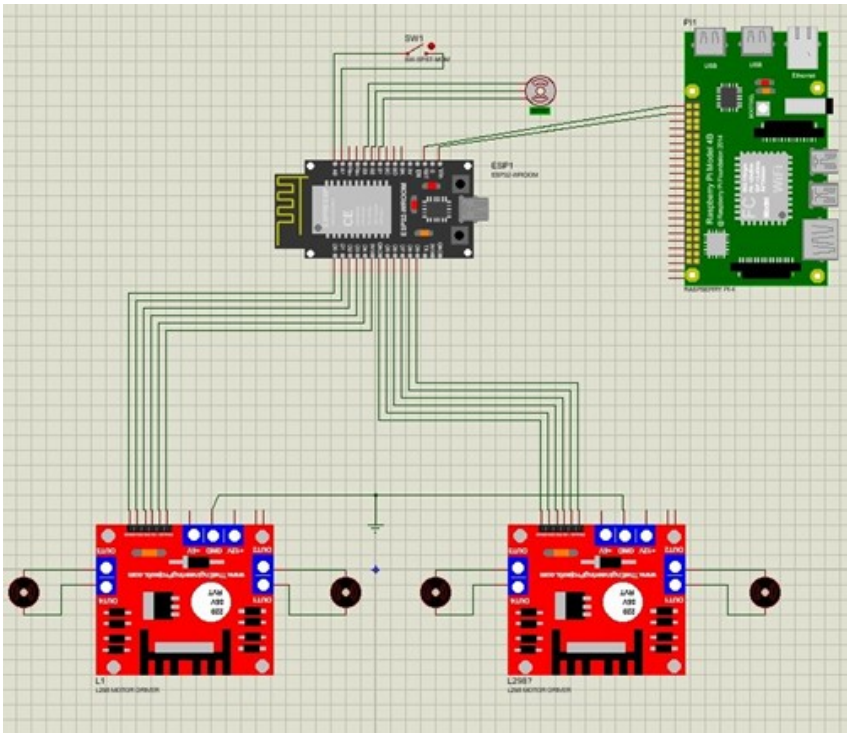

Figure 6: Wiring diagram showing the full electronics layout

9.2 SolidWorks 3D Design

The SolidWorks model below shows the physical design of the bin. The trash bin body is mounted on a square chassis that houses the electronics and the four Mecanum wheels, one at each corner. The Kinect camera is mounted on an arm extending from the front-left side of the bin at an angle that gives it a clear view of the throwing zone in front of the bin. The lid of the bin is controlled by a servo motor and sits on top of the cylindrical bin body. This design keeps the center of gravity low (motors and batteries in the base) while giving the Kinect a high enough vantage point to track thrown objects effectively.

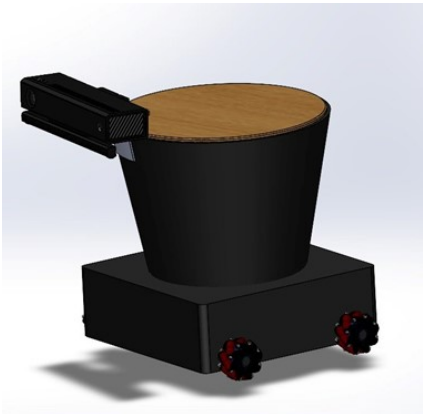

Figure 7: SolidWorks 3D model of the automated trash collector bin

10 Conclusion

This review covered the six papers that guided our design choices. From Annese et al. (2022) we learned that the Kinect camera gives reliable depth data at the ranges we need and integrates well with OpenCV. From Baduge et al. (2024) we chose YOLOv8 nano as our object detector for the idle phase on the Raspberry Pi. Zhang et al. (2020) gave us MediaPipe Hands for fast, GPU-free gesture recognition. Jacewicz et al. (2022) confirmed that a Kalman filter can accurately predict projectile landing points from noisy sensor data. Zeng et al. (2025) showed that AprilTag-based docking is reliable and accurate for mobile robots. And Shafiq et al. (2024) gave us the kinematic model and validation that four Mecanum wheels with DC motors and H-bridges can achieve the omnidirectional movement our bin needs. Additionally, our preliminary work with geometric transformations, intensity adjustments, Gaussian smoothing, and specialized detectors for heads and red objects has established a robust preprocessing foundation that will support the main tracking pipeline. The initial wiring diagram and SolidWorks model presented in Section 9 translate these findings into a concrete hardware design ready for prototyping.

References

[1] V. Annese, D. D'Ambruoso, and D. De Venuto, "Evaluating the accuracy of the azure kinect and kinect v2," *Sensors*, vol. 22, 2022.

[2] S. Baduge *et al.*, "Benchmarking deep learning models for object detection on edge computing devices," 2024.

[3] F. Zhang *et al.*, "Mediapipe hands: On-device real-time hand tracking," 2020.

[4] M. Jacewicz *et al.*, "Projectile trajectory estimation: Performance analysis of an extended kalman filter and an imperfect invariant extended kalman filter," *Defence Technology Journal*, 2022.

[5] Y. Zeng *et al.*, "Multi-sensor fusion for autonomous mobile robot docking: Integrating lidar, yolo-based apriltag detection, and depth-aided localization," *Electronics*, 2025.

[6] M. Shafiq, A. Imran, S. Maznoor, A. Majeed, and B. Ahmed, "Real-time navigation of mecanum wheel-based mobile robot in a dynamic environment," *Heliyon*, 2024.